

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

C05: *Design AI-based systems using PySpark, RDD operations, and intelligent agent architectures, using scalable systems and integrate components in distributed AI applications. (BTL 5)*

Assessment Tool Weightage: 50%

QUESTIONS: 5

Annexure A

Duration : 1 Hour
Total Marks:50

Design Task

Code of Conduct:

I, Nethra P(192511012) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Nethra P

Design Task

1. Hybrid AI Recommendation Engine Design

Design a hybrid AI-based recommendation system that integrates content-based and collaborative filtering techniques using PySpark RDDs. Explain how distributed data processing can be used to handle large-scale user-item interactions, and describe the role of intelligent agents in adapting recommendations based on user behavior and feedback in real time.

2. Smart Disaster Detection System

Develop a distributed AI system using PySpark and RDD operations to detect natural disasters (such as floods or earthquakes) from multi-source data streams (satellite images, sensors, and social media). Describe the system architecture, data fusion techniques, and how intelligent agents coordinate to provide early warnings and response strategies.

3. AI-Based Fraud Detection Framework

Design a scalable fraud detection system using PySpark RDDs to process large volumes of financial transactions. Explain how anomaly detection algorithms can be implemented in a distributed manner and how intelligent agents collaborate to identify suspicious patterns, trigger alerts, and continuously improve detection accuracy.

4. Autonomous Supply Chain Optimization System

Construct a distributed AI system using PySpark for optimizing supply chain operations (inventory, logistics, demand forecasting). Describe how RDD transformations support large-scale data processing and how multiple intelligent agents interact to make decentralized decisions for cost minimization and efficiency improvement.

5. Personalized Learning Analytics Platform

Design an AI-driven educational platform that uses PySpark and RDDs to analyze student learning behavior and performance data. Explain how the system supports adaptive learning through intelligent agents, real-time feedback, and scalable analytics to personalize content delivery for thousands of learners.

RUBRICS FOR CALCULATION

Evaluation Criteria	Problem Understanding & Requirement Analysis	System Architecture Design	Use of PySpark & RDD Operations	Intelligent Agent Integration	Scalability & Distributed Computing Design	Data Flow & Processing Pipeline	Innovation & Creativity	Clarity, Presentation & Documentation
Weightage	10%	20%	15%	15%	10%	10%	10%	10%

Criteria	Excellent (5)	Good (4)	Average (3)	Poor (2)
Problem Understanding & Requirement Analysis	Clearly defines problem, objectives, and constraints	Mostly clear with minor gaps	Basic understanding	Unclear or incorrect
System Architecture Design	Complete, scalable, well-structured architecture with diagrams	Good design with minor issues	Basic design, lacks detail	Poor/incomplete
Use of PySpark & RDD Operations	Efficient and correct use of RDD transformation s/actions	Mostly correct usage	Limited or partial use	Incorrect/missing
Intelligent Agent Integration	Strong integration with clear agent roles and logic	Good integration	Basic integration	Poor/no integration
Scalability & Distributed Computing Design	Clearly addresses scalability, fault tolerance, parallelism	Covers most aspects	Limited consideration	Not addressed
Data Flow & Processing Pipeline	Clear, logical, and well-defined data flow	Mostly clear	Some gaps	Poorly defined

Innovation & Creativity	Highly innovative, realistic, and impactful solution	Some innovation	Basic idea	No originality
Clarity, Presentation & Documentation	Very clear, neat diagrams, well-organized report	Mostly clear	Somewhat organized	Poor presentation

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence **COURSE CODE:** CSA17

CO5: Design AI-based systems using PySpark, RDD operations, and intelligent agent architectures, using scalable systems and integrate components in distributed AI applications. (BTL 5)

Assessment Tool Weightage: 50% Duration: 1 Hour Total Marks: 50

Question 1: Hybrid AI Recommendation Engine Design

1. Problem Understanding and Requirement Analysis

Modern e-commerce and media platforms must recommend items to millions of users in real time, using both what an item is about (content) and how similar users have behaved (collaborative signals). A single technique falls short: content-based filtering cannot capture social/trend signals and suffers from over-specialisation, while collaborative filtering suffers from the cold-start problem for new users or items and struggles with data sparsity. The objective of this design is therefore to build a hybrid recommendation engine that fuses content-based and collaborative filtering scores, computed in a distributed manner over PySpark RDDs, and continuously adapted through intelligent agents that observe user feedback in real time. Constraints include very large and constantly growing user-item interaction logs (views, clicks, ratings, purchases), the need for sub-second response at serving time, and the requirement that the system scale horizontally as the user base grows.

2. System Architecture

The architecture is organised into five layers:

- Data Ingestion Layer: Kafka/Flume streams collect clickstream, purchase, and rating events and land them in a distributed store (HDFS/S3); batch logs are also ingested periodically.
- Distributed Processing Layer (PySpark Core): Raw interaction logs and item metadata are loaded as RDDs and transformed into two parallel pipelines — a content-based feature pipeline and a collaborative filtering pipeline.
- Hybrid Fusion Layer: Scores from both pipelines are combined using a weighted linear hybrid or a switching/ stacking hybrid, producing a unified ranked list per user.
- Intelligent Agent Layer: Autonomous agents monitor user feedback (clicks, dwell time, skips) and dynamically re-weight the hybrid model, trigger retraining, and personalise ranking per user segment.
- Serving Layer: Precomputed top-N recommendations are cached in a low-latency store (Redis) and served via an API to the application front end.

3. Use of PySpark and RDD Operations

Interaction logs (userId, itemId, rating/implicit-signal, timestamp) are loaded as an RDD and processed with core RDD transformations and actions:

```
interactions_rdd = sc.textFile("hdfs:///data/interactions/*")
    .map(lambda line: line.split(","))
    .map(lambda x: (x[0], x[1], float(x[2])))

# Content-based: build item profile vectors (TF-IDF over item metadata)
item_profiles = item_meta_rdd.map(lambda x: (x[0], build_tfidf_vector(x[1])))

# Collaborative filtering: user-item matrix via groupByKey + reduceByKey
user_item_rdd = interactions_rdd.map(lambda x: ((x[0], x[1]), x[2]))
    .reduceByKey(lambda a, b: (a + b) / 2)
```

```
# Similarity computation distributed via cartesian / broadcast join
item_sim_rdd = item_profiles.cartesian(item_profiles)
                    .filter(lambda p: p[0][0] != p[1][0])
                    .map(lambda p: (p[0][0], (p[1][0], cosine_sim(p[0][1],
p[1][1]))))

hybrid_scores = cf_scores_rdd.join(content_scores_rdd)
                    .mapValues(lambda s: 0.6*s[0] + 0.4*s[1])
```

groupByKey/reduceByKey aggregate ratings per user-item pair, map/flatMap build feature vectors and expand candidate item sets, join combines the collaborative and content scores per user, and actions such as takeOrdered(N) extract the final top-N recommendation list per partition. Partitioning by userId keeps related data co-located, minimising shuffle during the join stage, and caching (persist(StorageLevel.MEMORY_AND_DISK)) is applied to the item-profile RDD since it is reused across many user computations.

4. Role of Intelligent Agents

A society of lightweight, cooperating agents governs adaptivity:

- Monitoring Agent: continuously observes streaming feedback (click-through, dwell time, cart-adds, skips) and computes an implicit reward signal per recommendation shown.
- Adaptation Agent: uses this reward signal in a contextual-bandit / reinforcement-learning loop to adjust the hybrid fusion weight (α) between content-based and collaborative scores for each user segment, favouring collaborative filtering for active users and content-based scoring for cold-start users.
- Retraining Agent: schedules incremental Spark jobs to refresh item similarity matrices and latent factor models when drift in feedback distribution is detected, avoiding a full batch retrain each cycle.
- Negotiation/Coordinator Agent: resolves conflicts when multiple business objectives (diversity, novelty, revenue) compete, using a multi-objective utility function to pick the final ranking.

This agent-based feedback loop allows the recommender to adapt in near real time to shifting user taste without waiting for the next full batch cycle.

5. Scalability and Distributed Computing Design

- Horizontal scaling: additional Spark executors are added as data volume grows; RDD partitions are tuned (typically 2–4x the number of cores) to balance load.
- Fault tolerance: RDD lineage (DAG) enables automatic recomputation of lost partitions without checkpointing every stage; checkpointing is used for iterative similarity computations to truncate long lineage chains.
- Parallelism: the cartesian/join heavy similarity computation is optimised using locality-sensitive hashing (LSH) to avoid an $O(n^2)$ all-pairs comparison at scale.
- Data skew handling: salting is applied to popular item keys before groupByKey/reduceByKey to avoid a small number of executors becoming bottlenecks.

6. Data Flow and Processing Pipeline

1. Raw events streamed into Kafka → landed in HDFS/S3 as immutable logs.
2. PySpark batch/streaming job loads logs into RDDs, cleans and deduplicates records.
3. Parallel content-based and collaborative pipelines compute per-item and per-user feature/similarity scores.
4. Hybrid fusion layer joins both score RDDs and computes the final weighted score.
5. Top-N recommendations per user are written to a low-latency cache for serving.
6. Feedback on served recommendations flows back into the ingestion layer, closing the loop for agent-driven adaptation.

7. Innovation and Impact

The design's innovation lies in coupling a bandit-driven agent layer with the hybrid Spark pipeline so that the fusion weight itself is learned per user rather than fixed globally, and in using LSH-based approximate similarity to make the content-based half of the system scale sub-linearly. This yields a system that remains accurate for cold-start users while scaling to millions of interactions per hour.

Question 2: Smart Disaster Detection System

1. Problem Understanding and Requirement Analysis

Natural disasters such as floods and earthquakes evolve quickly and are best detected by fusing heterogeneous, high-velocity data sources: satellite imagery, IoT sensor telemetry (water level, seismic vibration), and unstructured social-media text reporting eyewitness observations. The system must ingest and correlate these sources at scale, detect anomalies indicating a disaster onset within seconds to minutes, and dispatch early warnings before the event escalates — all while tolerating noisy, incomplete, and sometimes contradictory data streams.

2. System Architecture

- Multi-Source Ingestion Layer: satellite image feeds (via APIs), sensor telemetry (via MQTT/Kafka), and social-media streams (via Twitter/X streaming API) are ingested concurrently.
- Distributed Processing Layer (PySpark): Spark Streaming (structured streaming on RDD-based micro-batches) processes each source into normalized event RDDs.
- Data Fusion Layer: a fusion module aligns events by geolocation and timestamp, computing a combined disaster-probability score.
- Intelligent Agent Layer: detection, verification, and alert-dispatch agents coordinate to confirm events and trigger warnings.
- Response/Alerting Layer: confirmed alerts are pushed to emergency-response dashboards, SMS/public alert gateways, and government disaster-management systems.

3. Use of PySpark and RDD Operations

```
sensor_rdd = ssc.socketTextStream(host, port) \
    .map(lambda x: parse_sensor(x))           # (sensorId, geo, value, ts)

satellite_rdd = sc.binaryFiles("hdfs:///sat/tiles/*") \
    .map(lambda f: (f[0], extract_flood_signature(f[1])))

social_rdd = tweet_stream.filter(lambda t: is_disaster_related(t)) \
    .map(lambda t: (geo_bucket(t), t))

# Fuse by geo-bucket key using cogroup / join
fused_rdd = sensor_rdd.map(lambda x: (x[1], x)) \
    .cogroup(satellite_rdd, social_rdd) \
    .mapValues(compute_fusion_score)

alerts_rdd = fused_rdd.filter(lambda x: x[1] > THRESHOLD)
alerts_rdd.foreachRDD(lambda rdd: rdd.foreachPartition(dispatch_alert))
```

map/filter normalise and select disaster-relevant records per source; cogroup performs a distributed three-way join keyed by geographic bucket to fuse satellite, sensor, and social-media signals sharing the same location and time window; reduceByKey aggregates sensor readings within a bucket (e.g., average water level); and foreachPartition is used as an action to dispatch alerts directly from each executor without collecting all data to the driver, preserving distributed execution.

4. Data Fusion Techniques

- Spatio-temporal alignment: all three sources are bucketed into a common geohash grid and time window before fusion.
- Weighted evidence fusion: a Bayesian/Dempster–Shafer-style combination merges sensor readings (high reliability, low coverage), satellite imagery (medium reliability, wide coverage) and social-media mentions (low reliability, very wide coverage, useful for corroboration) into a single confidence score.
- Cross-validation: an event is only escalated to "confirmed" when at least two independent modalities agree, reducing false positives from noisy social-media chatter.

5. Role of Intelligent Agents

- Detection Agents (per-source): each source has a dedicated agent that runs lightweight anomaly detection (e.g., threshold/statistical control charts for sensors, CNN-based flood-signature detection for satellite tiles, NLP classification for social posts).
- Fusion/Verification Agent: aggregates outputs from detection agents, applies the fusion score, and decides whether evidence crosses the confirmation threshold.
- Coordination Agent: manages communication between geographically distributed detection agents (e.g., neighbouring river-basin sensors) to detect spatially propagating events such as a flood wave.
- Alert-Dispatch Agent: formats and routes warnings to the correct stakeholders (local authorities vs. national agencies) based on severity, and negotiates de-duplication of repeated alerts.

6. Scalability and Distributed Computing Design

- Micro-batch streaming with tunable batch interval allows the system to trade latency against throughput as sensor density grows.
- Data is partitioned by geohash so that fusion computations for one region do not depend on data from unrelated regions, enabling embarrassingly parallel execution.
- Checkpointing of streaming state ensures fault tolerance across executor failures without losing in-flight disaster evidence.
- Elastic executor allocation (dynamic allocation) scales resources up automatically during a disaster surge in data volume.

7. Data Flow and Processing Pipeline

1. Sensor, satellite, and social streams ingested in parallel via Kafka/MQTT/REST connectors.
2. Each stream is cleaned and geo-tagged by its dedicated PySpark streaming job.
3. cogroup-based fusion combines all three streams by geo-time bucket.
4. Verification agent confirms or rejects candidate events.
5. Confirmed alerts are dispatched to response systems and logged for post-event analysis.

8. Innovation and Impact

The innovation lies in treating each data modality as an independent intelligent agent whose confidence is combined through evidential fusion rather than a naive average, which substantially reduces false alarms from social-media noise while retaining the speed advantage of crowd-sourced reports — enabling early warnings potentially minutes ahead of purely sensor-based systems.

Question 3: AI-Based Fraud Detection Framework

1. Problem Understanding and Requirement Analysis

Financial institutions process millions of transactions per hour, and fraudulent patterns evolve continuously as fraudsters adapt to detection rules. The system must therefore process large transaction volumes in a distributed

manner, detect anomalies in near real time, minimise false positives (which harm customer experience) and false negatives (which cause financial loss), and continuously retrain to keep pace with concept drift in fraud tactics.

2. System Architecture

- Ingestion Layer: transaction streams from card networks/payment gateways enter via Kafka.
- Distributed Feature Engineering Layer (PySpark RDDs): computes per-user, per-merchant, and per-device behavioural features (velocity, geo-distance between consecutive transactions, amount z-scores).
- Anomaly Detection Layer: distributed unsupervised models (Isolation Forest, clustering) and supervised classifiers score each transaction.
- Intelligent Agent Layer: agents collaborate to correlate suspicious transactions across accounts, trigger alerts, and provide feedback for model improvement.
- Action Layer: confirmed fraud cases trigger transaction holds, step-up authentication, or case creation for human analysts.

3. Use of PySpark and RDD Operations

```
txn_rdd = sc.textFile("hdfs:///txns/stream/*") \
    .map(parse_txn)                                # (accId, ts, amount, merchant, geo)

# Distributed feature engineering per account, keyed groupByKey
acc_history_rdd = txn_rdd.map(lambda t: (t.accId, t)) \
    .groupByKey() \
    .mapValues(lambda txns: compute_velocity_features(txns))

# Broadcast a small merchant-risk lookup table to every executor
merchant_risk_bc = sc.broadcast(load_merchant_risk_table())
scored_rdd = txn_rdd.map(lambda t: (t, score_txn(t, merchant_risk_bc.value)))

anomaly_rdd = scored_rdd.filter(lambda x: x[1] > ANOMALY_THRESHOLD)
flagged_rdd = anomaly_rdd.reduceByKey(lambda a, b: max(a, b)) # dedupe per account
```

groupByKey aggregates a customer's transaction history for velocity-based feature computation; broadcast variables distribute a small merchant risk-score table to every executor to avoid an expensive shuffle join; map/filter score and threshold transactions in parallel; and reduceByKey consolidates multiple anomaly flags per account into a single case, avoiding duplicate alerts.

4. Distributed Anomaly Detection Algorithms

- Unsupervised: an Isolation Forest / distributed k-means clustering model is trained across partitioned data using Spark MLlib, flagging transactions that fall in low-density regions of feature space.
- Supervised: a gradient-boosted tree or logistic-regression classifier trained on historically labelled fraud cases scores new transactions; class imbalance is handled via distributed SMOTE-style oversampling or cost-sensitive loss.
- Graph-based: transactions are modelled as a graph (accounts, devices, merchants as nodes); distributed connected-component analysis over RDDs surfaces fraud rings sharing devices or IPs.

5. Role of Intelligent Agents

- Scoring Agent: applies the current anomaly model to each incoming transaction in real time.
- Correlation Agent: links flagged transactions across accounts/devices/merchants to detect coordinated fraud rings rather than isolated anomalies.
- Alerting Agent: prioritises and routes alerts to human analysts by severity, and can autonomously place low-risk holds pending review.


```
# Aggregate demand per SKU-warehouse-week using reduceByKey
demand_rdd = orders_rdd.map(lambda o: ((o.sku, o.warehouse, week(o.ts)), o.qty)) \
    .reduceByKey(lambda a, b: a + b)

# Join with current inventory levels (RDD join)
inventory_rdd = sc.textFile("hdfs:///scm/inventory/*").map(parse_inventory)
gap_rdd = demand_rdd.map(lambda x: ((x[0][0], x[0][1]), x[1])) \
    .join(inventory_rdd) \
    .mapValues(lambda v: v[0] - v[1]) # shortfall/surplus

reorder_rdd = gap_rdd.filter(lambda x: x[1] > REORDER_THRESHOLD)
```

reduceByKey aggregates order quantities into weekly demand per SKU per warehouse; RDD join combines forecast demand with live inventory to compute shortfall/surplus at each node; map/flatMap transform routing options into a candidate solution space; and filter/sortBy actions identify SKUs and routes needing intervention. Partitioning by warehouse ID localises computation, since decisions at one warehouse rarely depend directly on another's raw data.

4. RDD Transformations Supporting Large-Scale Processing

- groupByKey/reduceByKey: aggregate order and shipment volumes by SKU, warehouse, and time window.
- join/cogroup: combine demand forecasts, inventory snapshots, and transportation-cost tables that live in separate RDDs.
- mapPartitions: run per-partition optimisation solvers (e.g., a local linear-programming pass) without the overhead of per-record function calls.
- persist/cache: forecast and cost-table RDDs, reused across many downstream optimisation iterations, are cached in memory to avoid recomputation.

5. Role of Intelligent Agents in Decentralised Decision-Making

- Warehouse Agents: each warehouse has an autonomous agent that monitors local inventory, forecasts local demand, and decides when/how much to reorder.
- Transportation Agents: plan and optimise routes/carrier selection for outbound shipments, reacting to real-time traffic/weather data.
- Negotiation/Coordinator Agent: mediates when warehouse agents compete for limited supplier capacity or shared transportation resources, using auction-based or contract-net protocols to allocate resources fairly and efficiently.
- Learning Agent: monitors forecast accuracy and cost outcomes, feeding results back to retrain the demand-forecasting models and adjust agent decision policies over time.

6. Scalability and Distributed Computing Design

- Each warehouse/region operates its own agent and Spark job, enabling horizontal scale-out as new warehouses are added.
- The coordinator agent operates on a much smaller, aggregated summary RDD rather than raw transactional data, keeping the central negotiation step lightweight.
- Dynamic resource allocation lets Spark clusters expand during peak seasons (e.g., festive demand surges) and contract afterward.

7. Data Flow and Processing Pipeline

1. Order, inventory, and shipment data ingested continuously from ERP/IoT sources.
2. PySpark jobs aggregate demand and compute inventory gaps per SKU/warehouse.
3. Forecasting models predict near-term demand per region.
4. Warehouse and transportation agents negotiate replenishment and routing decisions via the coordinator agent.

5. Approved decisions are executed as purchase orders/shipment schedules; outcomes feed back for continuous learning.

8. Innovation and Impact

Framing supply-chain optimisation as a multi-agent negotiation problem — rather than a single centralised optimiser — allows the system to make fast, local decisions at each warehouse while still achieving globally efficient outcomes through lightweight coordination, improving resilience to disruption at any single node.

Question 5: Personalized Learning Analytics Platform

1. Problem Understanding and Requirement Analysis

Online learning platforms generate massive volumes of behavioural data — clickstreams, quiz attempts, video-watch patterns, and forum interactions — from thousands of learners simultaneously. The platform must analyse this data at scale to understand each learner's pace, strengths, and weaknesses, then adapt content delivery and feedback in near real time, all while remaining scalable as learner numbers and content catalogues grow.

2. System Architecture

- Data Collection Layer: learning-management-system events (video plays, quiz submissions, page navigation, time-on-task) stream into the platform.
- Distributed Analytics Layer (PySpark): RDD pipelines compute per-student engagement and performance metrics at scale.
- Adaptive Modelling Layer: student-mastery models (e.g., Bayesian Knowledge Tracing) estimate skill mastery per topic.
- Intelligent Agent Layer: tutoring/recommendation agents select the next best content item or intervention per learner.
- Delivery Layer: personalised content, hints, and real-time feedback are pushed back to the learner interface.

3. Use of PySpark and RDD Operations

```
events_rdd = sc.textFile("hdfs:///lms/events/*") \
    .map(parse_event)                                # (studentId, topicId, eventType, score,
ts)

# Aggregate performance per student-topic using groupByKey
perf_rdd = events_rdd.filter(lambda e: e.eventType == "QUIZ") \
    .map(lambda e: ((e.studentId, e.topicId), e.score)) \
    .groupByKey() \
    .mapValues(lambda scores: sum(scores)/len(scores))

# Engagement features via reduceByKey
engagement_rdd = events_rdd.map(lambda e: (e.studentId, e.duration)) \
    .reduceByKey(lambda a, b: a + b)

mastery_rdd = perf_rdd.join(engagement_rdd.map(lambda x: (x[0], x))) \
    .mapValues(compute_mastery_score)

at_risk_rdd = mastery_rdd.filter(lambda x: x[1] < RISK_THRESHOLD)
```

groupByKey/mapValues aggregate quiz scores per student per topic into an average performance metric; reduceByKey sums engagement time per student across sessions; join combines performance and engagement RDDs to compute a composite mastery score; and filter isolates at-risk students for early intervention. Partitioning by studentId ensures each learner's full history stays co-located, minimising cross-executor shuffle during aggregation.

4. Adaptive Learning through Intelligent Agents

- Diagnostic Agent: continuously updates each learner's mastery profile from incoming quiz and interaction data.
- Tutoring Agent: selects the next content item (easier remedial material, standard progression, or enrichment) based on the learner's current mastery level and learning-style signals, effectively acting as a personalised policy per student.
- Early-Warning Agent: flags learners falling behind pace or showing disengagement (long inactivity, repeated low scores) and triggers instructor notifications or automated nudges.
- Feedback Agent: delivers real-time, contextual feedback (hints, worked examples) during quiz attempts rather than only after completion.

5. Scalability and Distributed Computing Design

- Partitioning by studentId/topicId enables the system to process thousands of learners' streams in parallel without cross-partition dependencies for most aggregations.
- Incremental/streaming Spark jobs update mastery scores as new events arrive, rather than recomputing from the full history each time.
- Caching of static content-metadata RDDs (topic difficulty, prerequisite graphs) avoids repeated disk reads across millions of per-student computations.
- Horizontal scaling of Spark executors accommodates enrolment spikes (e.g., start of a semester) without redesigning the pipeline.

6. Data Flow and Processing Pipeline

1. Learner interaction events stream from the LMS into the ingestion layer.
2. PySpark jobs aggregate quiz performance and engagement metrics per student-topic.
3. Mastery models combine these into a per-topic mastery score for each learner.
4. Tutoring and early-warning agents use mastery scores to personalise the next content item or trigger interventions.
5. Personalised content/feedback is delivered to the learner interface; new interactions feed back into the pipeline, closing the adaptive loop.

7. Innovation and Impact

Coupling distributed mastery estimation with an agent-driven tutoring policy allows the platform to personalise content delivery for thousands of learners simultaneously and in near real time, rather than relying on static, one-size-fits-all course sequencing — improving both engagement and learning outcomes at scale.

Rubrics for Calculation

Evaluation Criteria	Weightage	Excellent (5)	Good (4)	Average (3)	Poor (2)
Problem Understanding & Requirement Analysis	10%	Clearly defines problem, objectives, and constraints	Mostly clear with minor gaps	Basic understanding	Unclear or incorrect
System Architecture Design	20%	Complete, scalable, well-structured architecture with diagrams	Good design with minor issues	Basic design, lacks detail	Poor/incomplete

Evaluation Criteria	Weightage	Excellent (5)	Good (4)	Average (3)	Poor (2)
Use of PySpark & RDD Operations	15%	Efficient and correct use of RDD transformations/actions	Mostly correct usage	Limited or partial use	Incorrect/missing
Intelligent Agent Integration	15%	Strong integration with clear agent roles and logic	Good integration	Basic integration	Poor/no integration
Scalability & Distributed Computing Design	10%	Clearly addresses scalability, fault tolerance, parallelism	Covers most aspects	Limited consideration	Not addressed
Data Flow & Processing Pipeline	10%	Clear, logical, and well-defined data flow	Mostly clear	Some gaps	Poorly defined
Innovation & Creativity	10%	Highly innovative, realistic, and impactful solution	Some innovation	Basic idea	No originality
Clarity, Presentation & Documentation	10%	Very clear, neat diagrams, well-organized report	Mostly clear	Somewhat organized	Poor presentation